Casa del Arte: Migración de un Recorrido 3D en Android a una Experiencia Inmersiva de Realidad Virtual con Google Cardboard SDK

Gerardo Assael Amaya Niño*, Nicolas Luevano Rodríguez*,
Mariana Abigail Olvera Zuñiga*, Angel Guadalupe Rivera Portillo*,
Manuel Alejandro Zapata Morales,*
*Universidad Politécnica de Victoria
Ingeniería en Tecnologías de la Información
Ciudad Victoria, Tamaulipas, México

***Resumen*—Este trabajo presenta la evolución de *Casa del Arte*, un recorrido tridimensional para Android de un recinto cultural, hacia una experiencia inmersiva de realidad virtual (VR) construida sobre el Google Cardboard SDK. El proyecto original renderizaba la escena con OpenGL ES 2.0 sobre una `Activity` y un `GLSurfaceView` convencionales, simulando un efecto estereoscópico de manera manual mediante la duplicación de la misma vista de cámara en dos mitades de la pantalla, sin seguimiento de cabeza ni separación interpupilar real. La migración a VR reemplazó dicho mecanismo por la infraestructura oficial de Cardboard (`CardboardActivity`, `CardboardView.StereoRenderer`), incorporando seguimiento de cabeza basado en sensores, corrección de distorsión de lente y cálculo correcto de las matrices de vista y proyección por cada ojo, además de retroalimentación háptica al activar el disparador del visor. Se documenta que toda la geometría procedural de la escena (prismas y planos con textura que conforman paredes, ventanas, puertas, techos, jardines, banquetas y portón) se conservó sin modificación alguna durante la migración, concentrándose el trabajo de adaptación exclusivamente en la capa de renderizado estereoscópico y en la integración con los sensores del dispositivo. Se describen la arquitectura resultante, un comparativo puntual entre ambas versiones del proyecto, y las limitaciones identificadas durante la revisión del código.**

I. Introducción

Casa del Arte nació como un proyecto previo de recorrido tridimensional para Android, desarrollado en un curso anterior de programación móvil, cuyo objetivo era modelar y explorar en primera persona un recinto cultural compuesto por una fachada principal, un edificio con múltiples habitaciones, jardines, banquetas, calle perimetral y un portón de acceso. Dicha versión inicial ya incorporaba, de manera incipiente, la idea de observar la escena a través de un visor tipo Cardboard: la aplicación dibujaba la escena completa dos veces por cuadro y ubicaba cada copia en una mitad de la pantalla, buscando imitar el efecto de dos imágenes lado a lado propio de estos visores. Sin embargo, ambas copias utilizaban exactamente la misma cámara y la misma matriz de proyección, por lo que no existía separación interpupilar real ni, sobre todo, ningún tipo de seguimiento de la orientación de la cabeza del usuario: la vista solo cambiaba en respuesta a un control de videojuegos.

El presente trabajo documenta la migración de esa versión hacia una experiencia de realidad virtual (VR) genuina [1], adoptando el Google VR SDK (Cardboard SDK) para Android. Este SDK provee la infraestructura necesaria para el seguimiento de la orientación de la cabeza mediante los sensores del teléfono [2, 3], la corrección de la distorsión introducida por las lentes del visor y el cálculo correcto de las matrices de vista y proyección para cada ojo, dejando al equipo desarrollador únicamente la tarea de construir el contenido 3D de la experiencia [4].

El objetivo de esta migración fue conservar íntegramente el contenido 3D ya construido en la versión previa (la totalidad de la geometría procedural del recinto, generada con primitivas propias y dibujada mediante OpenGL ES 2.0 [5, 6, 7]) y sustituir únicamente la capa de renderizado y de captura de movimiento por la infraestructura oficial de Cardboard, de modo que la vista respondiera de forma natural a los movimientos reales de la cabeza del usuario y no exclusivamente a un control físico. Dicho control tipo Xbox (botones, D-pad y joysticks analógicos), ya presente en la versión original conforme a los lineamientos de manejo de controladores de juego de Android [8], se conservó como método de entrada complementario para desplazarse dentro del recinto.

II. Desarrollo Experimental

II-A. Punto de partida: el recorrido 3D original

La versión previa de *Casa del Arte* se ejecutaba dentro de una `Activity` estándar de Android que alojaba un `GLSurfaceView` configurado en modo de renderizado continuo, y su clase `CubeRenderer` implementaba la interfaz genérica `GLSurfaceView.Renderer` (con los métodos clásicos `onSurfaceCreated`, `onSurfaceChanged` y `onDrawFrame`). Dentro de `onDrawFrame`, el efecto de visor Cardboard se simulaba de forma completamente manual: la misma cámara en primera persona se utilizaba para dibujar la escena completa dos veces en el mismo cuadro, una vez con el *viewport* de OpenGL fijado a la mitad izquierda de la pantalla y otra vez fijado a la mitad derecha (`glViewport`),

repitiendo literalmente la misma secuencia de llamadas a los métodos de dibujo (`piso`, `cielo_techo`, `laterales`, `banqueta`, `piso_casa`, `fachada_principal`, `esqueleto_edificio`, `jardines`, `porton`) para ambas mitades. Al no existir ningún desplazamiento (*offset*) entre las dos vistas, las dos imágenes resultantes eran idénticas: el usuario percibía una separación visual de la pantalla en dos mitades, pero no una estereoscopía real ni una sensación de profundidad, y el punto de vista solo se modificaba mediante los botones, el D-pad o los joysticks de un control físico tipo Xbox, sin ningún tipo de seguimiento de la orientación de la cabeza.

II-B. Contenido 3D reutilizado sin cambios

Al comparar el código fuente de ambas versiones del proyecto, se constató que la totalidad de las clases responsables de construir y dibujar la geometría del recinto se conservaron sin ninguna modificación funcional durante la migración (el único cambio detectado en dichos archivos fue el nombre del paquete de Java): `Prisma`, `Pared`, `Ventana`, `Piso`, `Jardin`, `Calle`, `Escalones`, `Porton`, `Banqueta`, `EsqueletoEdificio` y `FachadaPrincipal`, así como las utilidades `TextureHelper`, `ShaderHelper`, `RawResourceReader` y `Dpad`. En estas clases se generan de manera procedural, mediante primitivas propias [9], todos los elementos del recinto:

- `Prisma`: caja rectangular texturizada con sus seis caras, utilizada como bloque de construcción básico (columnas, escalones, tramos del portón, techo a dos aguas).
- `Pared` y `Ventana`: planos rectangulares con textura y transparencia habilitada (`GL_BLEND`), utilizados tanto para muros como, en el caso de `Ventana`, reutilizados para representar puertas mediante distintos parámetros de escala y textura.
- `Piso` y `Jardin`: planos grandes con coordenadas de textura repetidas (*tiling*) para simular pasto y piso de mosaico.
- `Calle`: un marco hueco compuesto de cuatro tramos rectangulares, reutilizado tanto para representar la calle perimetral como el techo del edificio principal (`EsqueletoEdificio`), aprovechando la misma geometría con distintos parámetros de tamaño.

Sobre estas primitivas se construyen las clases de composición de más alto nivel: `EsqueletoEdificio` (paredes, ventanas y puertas del edificio principal, incluyendo separadores de habitaciones), `FachadaPrincipal` (columnas, techo a dos aguas y decoraciones de la entrada), `Escalones` (una serie de prismas que se angostan progresivamente para simular una escalinata), `Porton` (estructura de acceso vehicular/peatonal con barras y postes) y `Banqueta` (superficie hueca tipo marco con bordillos laterales). Cada una de estas clases recibe identificadores de textura y dimensiones en su constructor, y expone un método `draw` que recibe la matriz de modelo vigente junto con los *handles* de OpenGL, aplicando internamente traslaciones y rotaciones mediante `android.opengl.Matrix` para posicionar cada sub-elemento. Esto permitió que el esfuerzo de migración se concentrara por completo en la capa de renderizado y de entrada, sin necesidad de rehacer el modelo 3D del recinto.

II-C. Migración a realidad virtual con Google Cardboard SDK

La adaptación a VR consistió en sustituir la infraestructura genérica de OpenGL de Android por la infraestructura oficial del Google Cardboard SDK, tanto en `MainActivity` y en `CubeRenderer`. La Tabla I resume los cambios más relevantes identificados entre ambas versiones.

En `MainActivity`, el cambio de `extends Activity` a `extends CardboardActivity` habilitó el método `onCardboardTrigger`, sobrescrito para vibrar el dispositivo mediante un `Vibrator` y reiniciar la posición de la cámara cada vez que el usuario presiona el botón físico del visor. En `CubeRenderer`, el antiguo bucle manual de doble dibujado fue remplazado por tres métodos del ciclo de vida de `StereoRenderer`: `onNewFrame` (sin operaciones adicionales), `onDrawEye(Eye eye)` (invocado una vez por cada ojo, donde se combina la matriz de vista del ojo obtenida del SDK, `eye.getEyeView()`, con la matriz de vista propia derivada del control físico, y se multiplica por la matriz de perspectiva calibrada del ojo, `eye.getPerspective()`, antes de invocar la misma secuencia de dibujado ya usada en la versión original) y `onFinishFrame`/`onRendererShutdown` (sin lógica adicional). El resto de la clase –las matrices de posición, los vectores `Position`, `ViewDir`, `RightVector` y `UpVector`, los métodos `MoveForward`, `MoveUpward`, `StrafeRight`, `RotateX`, `RotateY` y los límites de movimiento– permaneció sin cambios respecto a la versión previa.

II-D. Redistribución de puertas interiores y rotulación dinámica

Como parte de las mejoras a la estructura del recinto en `EsqueletoEdificio`, se reestructuró la distribución de las puertas hacia el patio interior. Se implementó una disposición de exactamente 26 puertas distribuidas en las cuatro paredes perimetrales del patio: 5 en el muro frontal (manteniendo libre el espacio central dedicado al portón de acceso), 7 en el muro trasero, 7 en el muro izquierdo y 7 en el muro derecho. Para garantizar que la totalidad de las puertas fueran visibles sin colisionar ni quedar ocultas dentro de las esquinas o muros exteriores del recinto, se ajustó la escala del ancho de las puertas internas (reducida a una proporción de 0.5) y se recalcularon sus coordenadas en los ejes X y Z.

Adicionalmente, se desarrolló la clase `TextTextureHelper`, la cual genera dinámicamente en tiempo de ejecución texturas de OpenGL ES 2.0 a partir de mapas de bits de Android (`Bitmap`, `Canvas` y `Paint`). Dicha clase renderiza placas indicadoras con fondo amarillo/dorado (`#FFD700`), doble borde decorativo y tipografía roja en negrita (`#CC0000`), las cuales se ubican a un costado de su respectiva entrada.

Tabla I: Comparativo entre la versión 3D original y la versión de realidad virtual

Aspecto	**Versión original (3D, sin VR real)**	**Versión con Google Cardboard SDK**
Actividad base	`Activity` + `GLSurfaceView`	`CardboardActivity` + `CardboardView`
Interfaz del renderer	`GLSurfaceView.Renderer` (`onSurfaceCreated`, `onSurfaceChanged`, `onDrawFrame`)	`CardboardView.StereoRenderer` (`onSurfaceCreated`, `onSurfaceChanged`, `onNewFrame`, `onDrawEye`, `onFinishFrame`, `onRendererShutdown`)
Generación de las dos vistas	Manual: la escena se dibuja dos veces con `glViewport` fijado a cada mitad de la pantalla, usando la misma cámara en ambas	Automática: el SDK invoca `onDrawEye(Eye eye)` una vez por cada ojo
Separación interpupilar / paralaje	Inexistente (ambas mitades son la misma imagen)	Calculada por el SDK mediante `eye.getEyeView()` y `eye.getPerspective()`
Seguimiento de cabeza	No implementado; la vista solo cambia con el control físico	Implementado mediante `HeadTracker`, `OrientationEKF`, `MagnetSensor` y `SensorEventProvider` del SDK
Corrección de distorsión de lente	No implementada	Implementada mediante `DistortionRenderer` y `Distortion`
Retroalimentación háptica	No implementada	Vibración de 50 ms al presionar el disparador del visor (`onCardboardTrigger`)
Control físico (Xbox/D-pad/joystick)	Único método de navegación	Conservado como método de entrada complementario al seguimiento de cabeza
Geometría de la escena (`Prisma`, `Pared`, `Ventana`, `Piso`, `Jardin`, `Calle`, `Escalones`, `Porton`, `Banqueta`, `EsqueletoEdificio`, `FachadaPrincipal`)	Implementación original	Idéntica, sin cambios funcionales

II-E. Interacción resultante

Tras la migración, la navegación dentro del recinto combina dos fuentes de entrada: el seguimiento real de la orientación de la cabeza del usuario, provisto por los sensores del teléfono a través del SDK de Cardboard, y el control físico tipo Xbox (botones `A`, `B`, `X`, `Y`, `START`, D-pad y joysticks analógicos), que se mantiene como mecanismo de desplazamiento y de reinicio de la posición de la cámara. El disparador del visor Cardboard, además, activa la vibración del dispositivo y reinicia la posición de la cámara, retroalimentación que no existía en la versión original del proyecto.

III. Resultados

El producto final de la migración es una aplicación Android completamente funcional que ejecuta el mismo recorrido arquitectónico del proyecto original, pero ahora dentro de un visor estereoscópico tipo Google Cardboard. Para dimensionar el cambio conviene partir de la versión previa. La Figura 1 corresponde al proyecto 3D original: la escena se dibujaba dos veces sobre la pantalla, pero ambas mitades empleaban la misma cámara, sin separación interpupilar ni corrección de lente, por lo que en la práctica se trataba de una vista monoscópica duplicada y no de una imagen apta para un visor.

Tras la migración, la aplicación arranca en una pantalla de bienvenida que presenta el mapeo de controles del mando físico e identifica a los desarrolladores del equipo. La Figura 2 muestra esta pantalla ya renderizada en modo Cardboard: la imagen se divide en dos vistas independientes, una por ojo, cada una con la deformación de barril necesaria para compensar la óptica de las lentes del visor.

Durante el recorrido, el usuario navega por el patio, los jardines y la fachada del edificio orientando la cabeza (seguimiento por los sensores del teléfono) y desplazándose con el

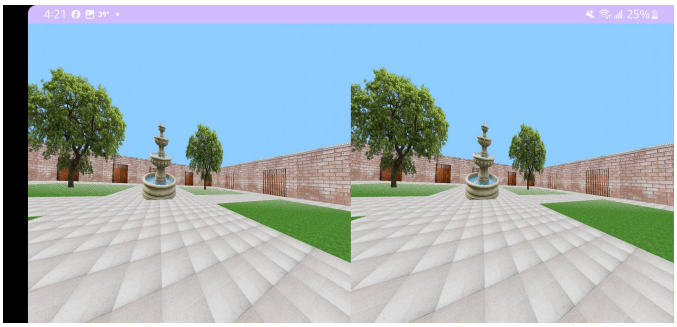


Figura 1: Versión previa a la migración. La escena se renderiza dos veces con la misma cámara, sin separación interpupilar ni distorsión de barril, produciendo un “pseudo-estéreo” que no aprovecha un visor real.

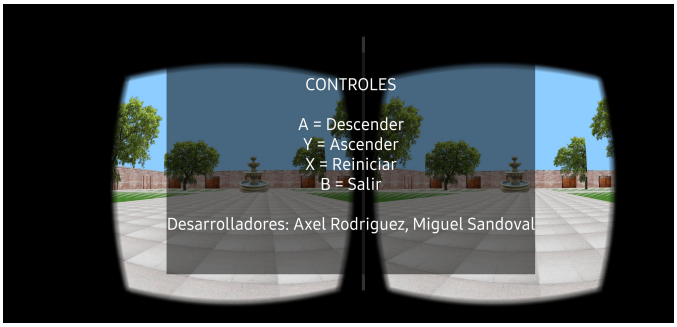


Figura 2: Pantalla de bienvenida y controles en modo VR. Cada ojo recibe una vista propia con distorsión de barril; el menú superpone el mapeo de botones (A=Descender, Y=Ascender, X=Reiniciar, B=Salir).

mando. La Figura 3 captura un momento del recorrido por la plaza central: se aprecia la separación interpupilar real entre ambas vistas y la curvatura característica de la corrección de lente, elementos ausentes en la versión original.

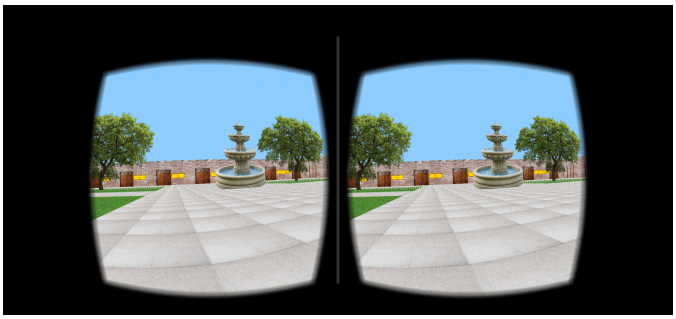


Figura 3: Recorrido inmersivo por la plaza central del modelo en modo VR. Ambas vistas presentan separación interpupilar y distorsión de barril, listas para un visor Cardboard.

En conjunto, los resultados verificables de la migración son:

- Renderizado estéreo real (una cámara por ojo) con separación interpupilar, frente a la vista duplicada de la versión original.
- Corrección de distorsión de lente (barril) aplicada por ojo, requisito indispensable para un visor Cardboard.
- Navegación por orientación de la cabeza mediante los sensores del dispositivo, además del control con mando físico ya existente.
- Reutilización íntegra de la geometría y las texturas del proyecto original, sin pérdida de contenido visual en la escena.

Como parte de las correcciones solicitadas por el instructor, se verificó y corrigió la cantidad de puertas que hay en la demo, añadiendo sus respectivos nombres de acuerdo a la sala que corresponde. La Figura 4 ilustra este resultado, mostrando los accesos a las salas de "Dirección" y "Fotografía, Oratoria, Composición Literaria".

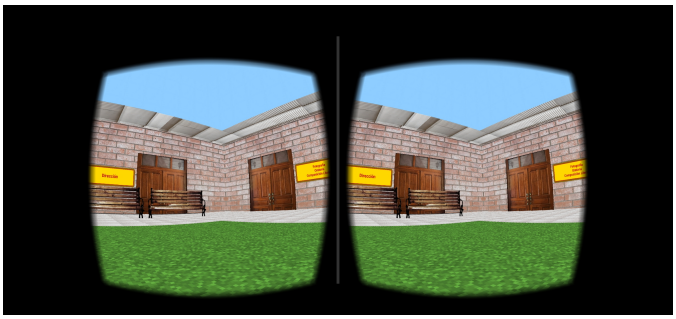


Figura 4: Vista inmersiva evidenciando la corrección en la cantidad de puertas y la asignación de sus respectivos nombres de acuerdo a la sala.

IV. Conclusión

La migración de *Casa del Arte* demuestra que es posible transformar un recorrido 3D convencional en una experiencia de realidad virtual genuina [10] sin reconstruir el contenido tridimensional ya existente: bastó con remplazar la capa de renderizado y de captura de movimiento –de una `Activity` con `GLSurfaceView` y un mecanismo manual de doble dibujado por mitades de pantalla, a una `CardboardActivity` con `CardboardView.StereoRenderer`– para pasar de una simulación superficial de estereoscopía, sin separación interpupilar ni seguimiento de cabeza, a una experiencia con seguimiento real de la orientación del usuario, corrección de distorsión de lente y retroalimentación háptica, todo ello provisto por el Google Cardboard SDK. La totalidad de la geometría procedural del recinto (prismas y planos con textura que conforman paredes, ventanas, puertas, techos, jardines, banquetas y portón) se conservó intacta, lo que confirma que la separación original entre la lógica de construcción de la escena y la lógica de renderizado facilitó considerablemente esta migración.

No obstante, la revisión del código evidenció áreas de mejora relevantes para futuras iteraciones del proyecto: eliminar el código muerto detectado en `TextureHelper` (heredado sin cambios desde la versión original), depurar los recursos de imagen sin uso, resolver por completo los límites de desplazamiento en los ejes X y Z –pendiente señalado por el propio equipo desde antes de la migración–, y reorganizar el proyecto conforme a la estructura estándar de Android Studio para facilitar su compilación, mantenimiento y colaboración entre los integrantes del equipo.

Referencias

[1] William R. Sherman y Alan B. Craig. *Understanding Virtual Reality: Interface, Application, and Design*. 2.ª ed. Cambridge, MA: Morgan Kaufmann, 2018.

[2] Steven M. LaValle. *Virtual Reality*. Cambridge, UK: Cambridge University Press, 2023.

[3] Google. *Motion Sensors — Android Developers*. 2024. URL: https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion (visitado 05-07-2026).

[4] Google LLC. *Google VR SDK for Android (Cardboard)*. 2019. URL: https://github.com/googlevr/gvr-android-sdk (visitado 05-07-2026).

[5] Khronos Group. *OpenGL ES 2.0 Specification*. 2010. URL: https://www.khronos.org/registry/OpenGL/specs/es/2.0/es_full_spec_2.0.pdf (visitado 05-07-2026).

[6] Android Developers. *OpenGL ES*. 2023. URL: https://developer.android.com/develop/ui/views/graphics/opengl (visitado 05-07-2026).

[7] Khronos Group. *OpenGL ES Common Profile Specification, Version 2.0*. Inf. téc. Khronos Group, 2010.

[8] Android Developers. *Handle Controller Actions*. 2023. URL: https://developer.android.com/training/game-controllers/controller-input (visitado 05-07-2026).

[9] Aaftab Munshi, Dan Ginsburg y Dave Shreiner. *OpenGL ES 2.0 Programming Guide*. Boston, MA: Addison-Wesley, 2008.

[10] Mel Slater y Sylvia Wilbur. “A Framework for Immersive Virtual Environments (FIVE): Speculations on the Role of Presence in Virtual Environments”. En: *Presence: Teleoperators and Virtual Environments* 6.6 (1997), págs. 603-616.